

Lab Practice Sheet 1

CS G526: Advanced Algorithms & Complexity

Instructor: Tulasimohan Molli

Date: 2026-09-06

! Evaluation Instructions for Students

- **Practice all 8 problems below.**
- In the next lab session, **one question will be chosen at random** for on-the-spot coding and evaluation.
- Your code will be evaluated by running it against automated test cases.
- You may use **Python**; **C++** and **Java**.

Q1: The Student Merit System

Description: Define a struct/class/object for **Student** with fields: **ID** (integer), **Name** (string), and **Marks** (3 subject scores). Write a program to:

1. Read details for N students.
2. Calculate the average marks for each student.
3. Print a “Merit List” of students whose average score is strictly above 75% in the order they appear in the input.
4. Identify and print the Topper (the student with the highest average score). If multiple students share the same highest average score, print **Topper: Name** for each of them on separate lines in the order they appear in the input.

• Input Format:

- Line 1: An integer N , the number of students.
- Next N lines: Each line contains an integer **ID**, a string **Name**, and 3 space-separated integers representing marks in three subjects.

• Output Format:

- First print **Merit List:** followed by lines of the form **ID Name (Avg: XX.XX)** for every student with average marks > 75.00 .
- Then print **Topper: Name** indicating the topper. If there is a tie for the highest average, print a **Topper: Name** line for each tied student in input order.

i Sample Input

```
2
101 Alice 80 90 85
102 Bob 60 50 55
```

i Sample Output

```
Merit List:
101 Alice (Avg: 85.00)
Topper: Alice
```

Q2: Matrix Multiplication

Description: Write a program to multiply two matrices A and B .

1. Read matrix A of dimensions $R_1 \times C_1$.
2. Read matrix B of dimensions $R_2 \times C_2$.
3. If multiplication is not possible ($C_1 \neq R_2$), output **Not Possible**.
4. Otherwise, compute and print the product matrix $C = A \times B$.

• **Input Format:**

- Line 1: Two space-separated integers R_1 and C_1 , representing dimensions of matrix A .
- Next R_1 lines: Each line contains C_1 space-separated integers representing the elements of matrix A .
- Next line: Two space-separated integers R_2 and C_2 , representing dimensions of matrix B .
- Next R_2 lines: Each line contains C_2 space-separated integers representing the elements of matrix B .

• **Output Format:**

- If $C_1 \neq R_2$, output a single line **Not Possible**.
- Otherwise, print R_1 lines, each containing C_2 space-separated integers representing the resulting product matrix $C = A \times B$.

i Sample Input 1

```
2 2
1 2
3 4
2 2
1 0
0 1
```

i Sample Output 1

```
1 2
3 4
```

i Sample Input 2

```
2 3
1 2 3
4 5 6
2 2
1 0
0 1
```

i Sample Output 2

Not Possible

Q3: Merge Two Sorted Arrays

Description: Given two sorted arrays A of size N_1 and B of size N_2 , merge them into a single sorted sequence in non-decreasing order using a linear two-pointer scan.

- **Input Format:**

- Line 1: An integer N_1 , the size of array A .
- Line 2: N_1 space-separated sorted integers representing array A .
- Line 3: An integer N_2 , the size of array B .
- Line 4: N_2 space-separated sorted integers representing array B .

- **Output Format:**

- A single line containing $N_1 + N_2$ space-separated integers representing the merged sorted sequence.

Sample Input

```
3
1 3 5
3
2 4 6
```

Sample Output

```
1 2 3 4 5 6
```

Q4: String Compression

Description: Given a string of lowercase alphabetic characters, compress consecutive repeating characters using run-length encoding. For example, consecutive occurrences of character c repeated k times should be represented as c followed by k .

- **Input Format:**

- A single line containing a string consisting of lowercase English letters.

- **Output Format:**

- A single line containing the compressed string.

i Sample Input

aaabbc

i Sample Output

a3b2c1

Q5: The Array Rotator

Description: Given an array of N integers, rotate the array by K positions.

- Direction $D = 0$ means **Left Rotation**.
- Direction $D = 1$ means **Right Rotation**.
- **Input Format:**
 - Line 1: An integer N , the size of the array.
 - Line 2: N space-separated integers representing the array.
 - Line 3: An integer $D \in \{0, 1\}$, where 0 indicates Left Rotation and 1 indicates Right Rotation.
 - Line 4: An integer K , the number of positions to rotate.
- **Output Format:**
 - A single line containing N space-separated integers representing the rotated array.

Sample Input

```
5
1 2 3 4 5
0
1
```

Sample Output

```
2 3 4 5 1
```

Q6: Segregate Even and Odd Values

Description: Given a sequence of N integers, rearrange the sequence so that all **even** numbers appear before all **odd** numbers, while strictly preserving their relative order from the original sequence (stable partition).

- **Input Format:**

- Line 1: An integer N , the number of elements.
- Line 2: N space-separated integers representing the sequence.

- **Output Format:**

- A single line containing N space-separated integers, with all even numbers listed first in original relative order, followed by all odd numbers in original relative order.

i Sample Input

```
7
17 15 8 12 10 5 4
```

i Sample Output

```
8 12 10 4 17 15 5
```

Q7: Insert After Pairs (Sum & Skip)

Description: Given a sequence of N integers, process the elements pairwise from left to right. After every pair of consecutive elements, insert a new element equal to their sum. If an odd element is left at the end without a pair, leave it as is.

- **Input Format:**

- Line 1: An integer N , the number of elements.
- Line 2: N space-separated integers representing the sequence.

- **Output Format:**

- A single line containing space-separated integers representing the modified sequence with sum elements inserted after each pair.

i Sample Input

```
4
2 4 5 3
```

i Sample Output

```
2 4 6 5 3 8
```

Q8: Move All Occurrences of Value X to the End

Description: Given a sequence of N integers and a target value X , rearrange the sequence such that all occurrences of X are moved to the end of the sequence, while the relative order of all other elements remains unchanged.

- **Input Format:**

- Line 1: An integer N , the number of elements.
- Line 2: N space-separated integers representing the sequence.
- Line 3: An integer X , the target value to be moved to the end.

- **Output Format:**

- A single line containing N space-separated integers representing the rearranged sequence.

i Sample Input

```
7
1 2 3 2 4 2 5
2
```

i Sample Output

```
1 3 4 5 2 2 2
```